
New algorithm for Half-plane Intersection and its Practical Value

— Thesis for Chinese Team Selecting
Contest 2006

By Zeyuan Zhu

Dedicate to my beloved mother, Xiaoli Xu.

New algorithm for Half-plane Intersection and its Practical Value

Zeyuan Zhu^①

Grade 12, Nanjing Foreign Language School, Jiangsu, China
2005-12-29

Keywords: Half-plane, Intersection, Feasible Region, Algorithm, Polygon, Practical

Abstract

Aim: Present a new $O(n\log n)$ algorithm for half-plane intersection (**abbr. HPI**), which is one of the most heatedly discussed problems in computer science; emphasize its advantages in practical application, and to some extent, reduce the complexity to $O(n)$. However, the new algorithm will be extraordinarily easy to be implemented.

§1 introduces what half-plane intersection is. §2 prepares a linear algorithm for convex polygon intersection (**abbr. CPI**). Equipped with such knowledge, a common solution for HPI is briefly discussed in §3. Then, my new algorithm emerges in §4 detailedly. Not only as a conclusion of the whole paper, §5 also discuss its further usage practically and compares it with the older algorithm described in §3.

Timestamps: Came up with it in April 2005; implemented partly in June 2005^②; problem set in July 2005^③; publicized as a post in USENET, November 6, 2005^④.

1. Introduction

A line in plane is usually represented as $ax+by=c$. Similarly, its inequality form $ax+by\leq c$ represents a *half-plane* (also named *h-plane* for short) as one side of this line. Notice that $ax+by\leq c$ and $-ax-by\leq -c$ show opposite h-planes unlike $ax+by=c$ and $-ax-by=-c$. Half-Plane Intersection (**abbr. HPI**) considers the following problem:

Given n half-planes, $a_ix+b_iy\leq c_i$ ($1\leq i\leq n$), you are to determine the set of all points that satisfying all the n inequations.

^① E-mail: zhuzeyuan[at]hotmail[dot]com

^② The sub-problem of HPI appeared in USA Invitational Computing Olympiad contest.

^③ Set an HPI problem in Peking University Online Judge, with brief introduction about the algorithm.

^④ URL: <http://groups.google.com/group/sci.math/msg/68e9d9fc634f0d92>

As **Figure i** describes, the feasible region, which is the intersection, forms a shape of convex hull but possibly unbounded. However, we shall add four h-planes forming a rectangle, which is large enough to make sure the area after intersections finite. *In the following sections, we suppose the feasible region is **bounded** with a finite area.*

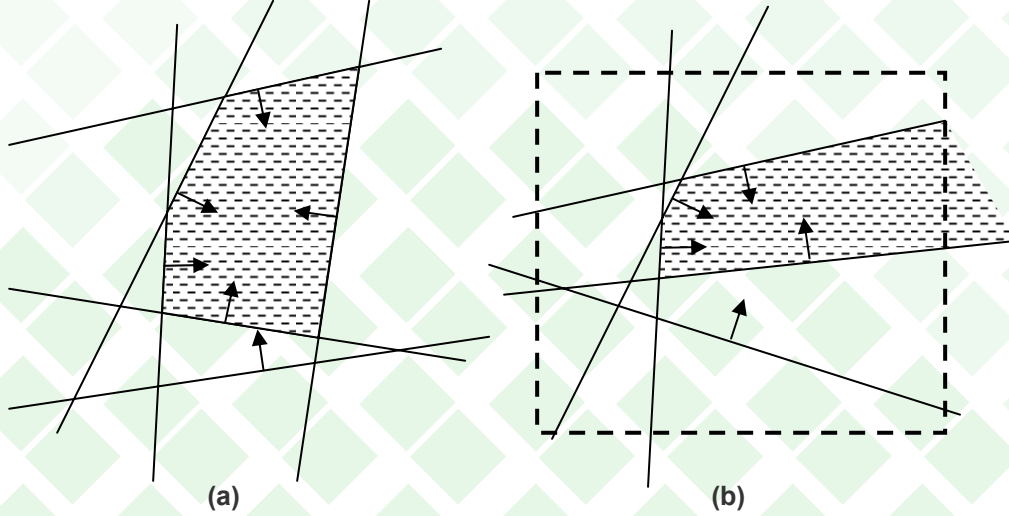


Figure i Two examples of HPI. (b) gives an example of unbounded intersection area.

Each h-plane builds up at most one side of the convex polygon, hence, the convex region will be bounded by at most n edges. Pay attention the intersection sometimes yields a line, a ray, a line-segment, a point or an empty region.

2. Convex Polygon Intersection (abbr. CPI)

Intersecting two convex polygons **A** and **B** into a single one, can be properly solved via Line Segment Intersection in $O(n \log n)$ time, when there are $O(n)$ edges. We will sketch out an easier and more efficient way, named *plane sweep method*.

The main idea is to calculate intersections of edges as cutting points, and break boundaries of **A** and **B**, into *outer edges* and *inner edges*. The segments of *inner edges* establish ties to each other, and form the shape of a polygon, which is the expected polygon after intersection. In **Figure ii**, *inner edges* are indicated by thick segments, which form a bold outline of the intersection.

Suppose there is a vertical sweep line, performing left-to-right sweep. The x-coordinates to be swept are called *x-events*. At anytime, there are at most four intersections from sweep line to either given polygon[®]:

✚ to the upper hull of **A** (name that intersection **Au** for short)

[®] Assume there is no edge in polygons parallel to the sweep line. If such situation happens, we could rotate the plane in proper angle, or else, we need good sense to judge a great many special cases.

- ✦ to the lower hull of **A** (name that intersection **Al** for short)
- ✦ to the upper hull of **B** (name that intersection **Bu** for short)
- ✦ to the lower hull of **B** (name that intersection **Bl** for short)

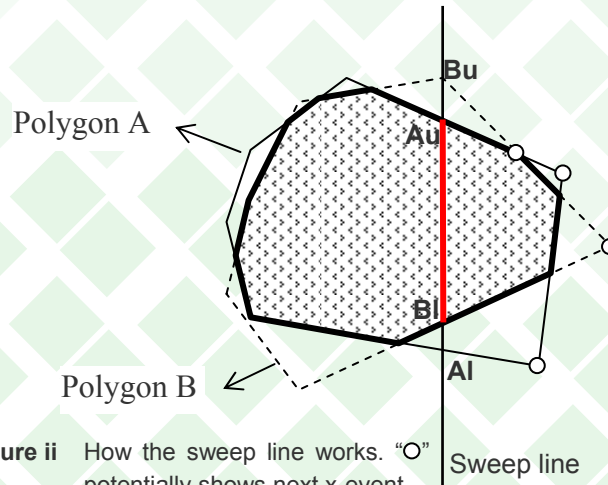


Figure ii How the sweep line works. “O” potentially shows next x-event.

Look at **Figure ii**, the lower one between intersections **Au** and **Bu**, and the upper one between intersections **Al** and **Bl**, form an interval of the current inner region – the red segment in bold.

Obviously, the sweep line may not go through all the x-event with rational coordinates. Call the edges where **Au**, **Al**, **Bu** and **Bl** are: ***e1***, ***e2***, ***e3*** and ***e4*** respectively. The next x-event should be chosen among four endpoints of ***e1***, ***e2***, ***e3*** and ***e4***, and four potential intersections: ***e1* ∩ *e3***, ***e1* ∩ *e4***, ***e2* ∩ *e3*** and ***e2* ∩ *e4***.

The above operation could be implemented with $O(n)$ running time, since there are $O(n)$ x-events, and the maintenance of **Au**, **Al**, **Bu** and **Bl** takes only $O(1)$.

3. Common solution: Divide-and-Conquer Algorithm (abbr. D&Q)

The basic approach is simple, depends on divide-and-conquer idea.

- ✦ **DIVIDE**: Partition the n h-planes into two sets of size $\lfloor \frac{n}{2} \rfloor$ and $\lceil \frac{n}{2} \rceil$.
- ✦ **CONQUER**: Compute the feasible region (intersection) recursively of both two sub-sets.
- ✦ **COMBINE**: Compute the intersection of two convex polygons, by linear CPI algorithm described in §2.

The total time complexity of the solution can be calculated via recursive equation:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = O(n \log n)$$

4. My New Solution: Sort-and-Incremental Algorithm (abbr. S&I)

Definition of h-plane's polar angle:

- ✦ for the h-plane like $x-y \geq \text{constant}$, we define its polar angle to $\frac{1}{4}\pi$.
- ✦ for the h-plane like $x+y \leq \text{constant}$, we define its polar angle to $\frac{3}{4}\pi$.
- ✦ for the h-plane like $x+y \geq \text{constant}$, we define its polar angle to $-\frac{1}{4}\pi$.
- ✦ for the h-plane like $x-y \leq \text{constant}$, we define its polar angle to $-\frac{3}{4}\pi$.

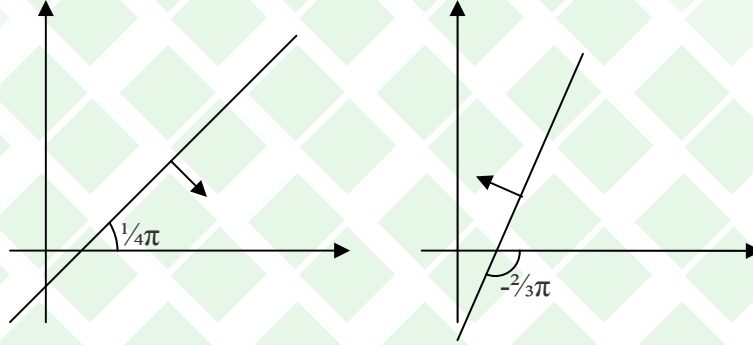


Figure iii Definition of half-plane's polar angle.

Definition of h-plane's constant:

- ✦ for the h-plane like $ax+by \leq c$, we say its constant is c .

My new *Sort-and-Incremental Algorithm* seems lengthy since I am going to introduce it in details:

Step 1: Separate the h-planes into two sets. One has polar angles of $(-\frac{1}{2}\pi, \frac{1}{2}\pi]$, the other has those of $(-\pi, -\frac{1}{2}\pi] \cup (\frac{1}{2}\pi, \pi]$.[®]

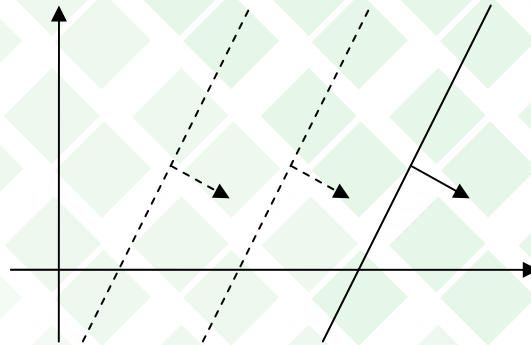


Figure iv For half-planes with the same polar angle, we must keep the one that is covered by others.

Step 2: Consider the h-plane set of polar angles in $(-\frac{1}{2}\pi, \frac{1}{2}\pi]$ (the other set should also go through step 3 and 4 similarly). Sort them by the polar angle. For the h-planes with the

[®] In fact, we only need to separate them into two parts, one contains $(a, a+\pi]$, the other contains $(a+\pi, a+2\pi]$. but my separation will make the explanation clearer

same polar angle, we can keep only one down (delete all others) according to the constant of these h-planes – see Figure iv above.

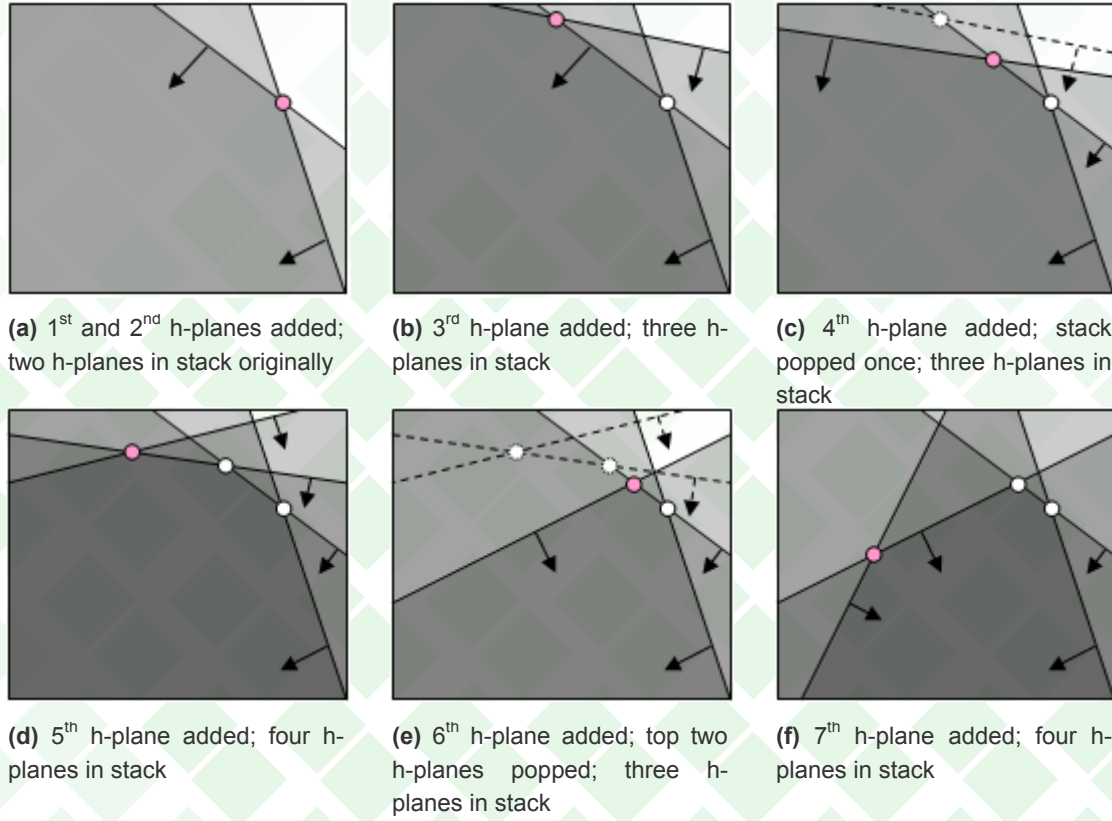


Figure v An example for **Step 3**. Old intersections to the left of the new ones are to be deleted. Dotted portion indicates the part to be deleted.

Step 3: Starting from two h-planes with the least polar angle, compute their intersection. Push them two into a stack.

Each time, add one more h-plane by increasing order of polar angles, and calculate the intersection of it and the top h-plane in stack. If this intersection is to the right of the intersection of top two h-planes in stack, we pop the stack once. Later, we check the intersection of two new top h-planes in stack... Do this repeatedly until it is to the left of the top intersection. At last, we push the current h-plane into the stack.

Step 4: The intersections of adjacent half-plane pairs in stack form half a convex polygon. For the two sets, we have two halves the convex polygon - $(-\frac{1}{2}\pi, \frac{1}{2}\pi]$ gives an upper hull and $(-\pi, -\frac{1}{2}\pi] \cup (\frac{1}{2}\pi, \pi]$ gives a lower hull. Merging them in linear time can be done via CPI in *section 2*, but we can make use of *iteration*.

Based on the knowledge of deque (double-ended queue), we repeatedly remove an end-point that is known to be infeasible. Look at Figure vi.

At the beginning, four pointers $p1$, $p2$, $p3$ and $p4$ indicate leftmost/rightmost edges of both upper and lower hulls. $p1$ and $p3$ move rightwards, while $p2$ and $p4$ walks leftwards.

At anytime, if the leftmost intersection is against the feasible region provided by $p1$ or $p3$, we are sure the leftmost one is to be removed. Naturally, $p1$ or $p3$ walks rightwards to its adjacent edge. The judgment holds analogously for rightmost intersection with $p2$ and $p4$.

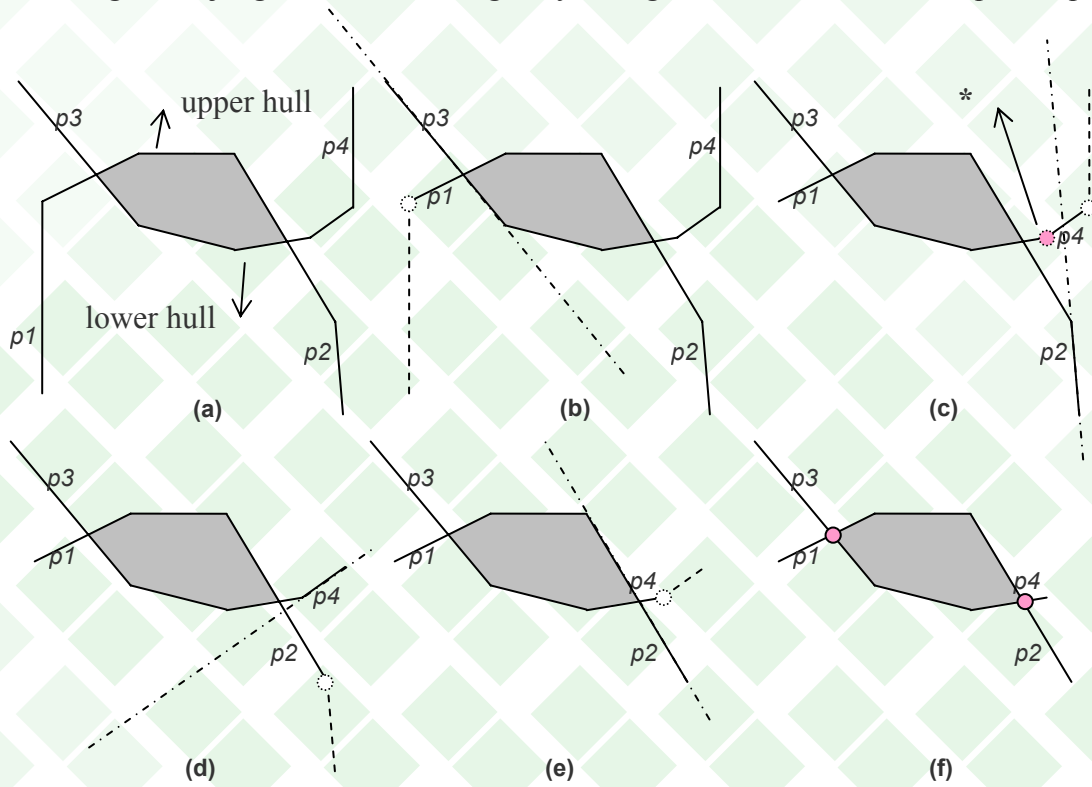


Figure vi An example for **Step 4**. In figure (c), only one intersection besides $p4$ can be removed, since the other (pink one marked with $*$) is in proper side of $p2$'s h-plane. That is why every time we try to remove only a single rightmost or leftmost intersection.

Do the above removing repeatedly until there is no more update. Meanwhile, the number of intersections for a single hull can decrease to zero, but still with non-empty feasible region:

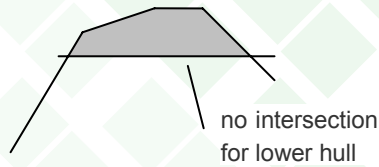


Figure vii Number of intersections for a single hull can decrease to zero.

Everything except sorting (Step 2) in S&I algorithm remain linear – $O(n)$ running time. Usually we use quick-sort. The total complexity is $O(n \log n)$, with fairly small constant factor hidden. **It can be reduced to $O(n)$, see the next section.**

5. Practical Value and Linear approach

Great ideas need landing gear as well as wings. S&I algorithm seems to work in the same time complexity as D&C algorithm, but some overwhelming advantages of implementing S&I holds:

- ✦ It is much easier to code S&I program than D&C one. The program in C++ programming language takes less than **3KB**.
- ✦ The coefficient hidden under S&I algorithm's complexity is extraordinarily smaller than D&C, since we no longer need $O(n \log n)$ number of intersection calculations. In general speaking, S&I program runs approximately five times faster than D&C one.

Remark: intersection calculations play the most important role in both algorithms due to its operational speed (so slow, equivalent to hundreds of addition operations). CPI, the preparative algorithm which will be called several times from D&C, requires $O(n)$ number of calculations, wherefore it rises the total running time up. Besides, S&I calculates $O(n)$ times in any case.

- ✦ If the given h-planes are all in $(-\frac{1}{2}\pi, \frac{1}{2}\pi]$ (or any span of π), S&I program can be shorten remarkably (to approximately **twenty** lines in C++), but D&C program may not. An informatics problem appeared in USA Invitational Computing Olympiad contest with such purpose.

Asymptotical Optimization to linear: The bottleneck of this algorithm is sorting. Pay attention the sorting is NOT a *comparison sort* (sorting based on comparison)! The key words for elements to be sorted are polar angles, which can be certainly determined by gradient – a decimal fraction. Since then, we can replace the $O(n \log n)$ quick-sort to $O(n)$ *radix-sort*. The asymptotical complexity of algorithm can decrease to $O(n)$. Anyway $O(n)$ approach usually runs slower than $n \log n$ ones for its additional memory usage!

The sentiment of my creation: An invention does not attribute to someone who comes up with ideas. Most people have ideas. The difference between the average person and the inventor is that the inventor for some reason believes only himself, and has the urge to see his ideas through to fruition. *Henri Matisse*, a French painter and sculptor, ever said 'there is nothing more difficult for a truly creative painter than to paint a rose, because before he can do so, he has first to forget all the roses that were ever painted.' Equipped with both idealistic and practical spirit of innovation, I am on the way. How about you?

Bibliography

1. Kurt Mehlhorn. *Data Structures and Algorithms Vol 3*. Springer-Verlag, New York, 1984.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to algorithms 2nd Edition*. MIT Press, New York, 2001
3. Gill Barequet. *Computational Geometry Lecture 4*, Spring 2004/2005.
4. Kavitha Telikepalli. *Algorithms and Data Structures, Lecture 3*, Winter 2003/2004.